



BRAINWARE UNIVERSITY

[PCC-CSM402]

[Design and Analysis of Algorithm]

Stages of algorithm development for solving a problem

The development of an algorithm (a plan) is a key step in solving a problem. Once we have an algorithm, we can translate it into a computer program in some programming language. Our algorithm development process consists of five major steps.

Step 1: Obtain a description of the problem.

Step 2: Analyze the problem.

Step 3: Develop a high-level algorithm.

Step 4: Refine the algorithm by adding more detail.

Step 5: Review the algorithm.

Step 1: Obtain a description of the problem.

This step is much more difficult than it appears. In the following discussion, the word *client* refers to someone who wants to find a solution to a problem, and the word *developer* refers to someone who finds a way to solve the problem. The developer must create an algorithm that will solve the client's problem.

The client is responsible for creating a description of the problem, but this is often the weakest part of the process. It's quite common for a problem description to suffer from one or more of the following types of defects: (1) the description relies on unstated assumptions, (2) the description is ambiguous, (3) the description is incomplete, or (4) the description has internal contradictions. These defects are seldom due to carelessness by the client. Instead, they are due to the fact that natural languages (English, French, Korean, etc.) are rather imprecise. Part of the developer's responsibility is to identify defects in the description of a problem, and to work with the client to remedy those defects.

Step 2: Analyze the problem.

The purpose of this step is to determine both the starting and ending points for solving the problem. This process is analogous to a mathematician determining what is given and what must be proven. A good problem description makes it easier to perform this step.

When determining the starting point, we should start by seeking answers to the following questions:

- What data are available?
- Where is that data?
- What formulas pertain to the problem?
- What rules exist for working with the data?
- What relationships exist among the data values?



BRAINWARE UNIVERSITY

[PCC-CSM402]

[Design and Analysis of Algorithm]

When determining the ending point, we need to describe the characteristics of a solution. In other words, how will we know when we're done? Asking the following questions often helps to determine the ending point.

- What new facts will we have?
- What items will have changed?
- What changes will have been made to those items?
- What things will no longer exist?

Step 3: Develop a high-level algorithm.

An algorithm is a plan for solving a problem, but plans come in several levels of detail. It's usually better to start with a high-level algorithm that includes the major part of a solution, but leaves the details until later. We can use an everyday example to demonstrate a high-level algorithm.

Problem: I need to send a birthday card to my brother, Mark.

Analysis: I don't have a card. I prefer to buy a card rather than make one myself.

High-level algorithm:

Go to a store that sells greeting cards
Select a card
Purchase a card
Mail the card

This algorithm is satisfactory for daily use, but it lacks details that would have to be added were a computer to carry out the solution. These details include answers to questions such as the following.

- "Which store will I visit?"
- "How will I get there: walk, drive, ride my bicycle, take the bus?"
- "What kind of card does Mark like: humorous, sentimental, risqué?"

These kinds of details are considered in the next step of our process.

Step 4: Refine the algorithm by adding more detail.

A high-level algorithm shows the major steps that need to be followed to solve a problem. Now we need to add details to these steps, but how much detail should we add? Unfortunately, the answer to this question depends on the situation. We have to consider who (or what) is going to implement the algorithm and how much that person (or thing) already knows how to do. If someone is going to purchase Mark's birthday card on my behalf, my instructions have to be adapted to whether or not that person is familiar with the stores in the community and how well the purchaser knows my brother's taste in greeting cards.



BRAINWARE UNIVERSITY

[PCC-CSM402]

[Design and Analysis of Algorithm]

When our goal is to develop algorithms that will lead to computer programs, we need to consider the capabilities of the computer and provide enough detail so that someone else could use our algorithm to write a computer program that follows the steps in our algorithm. As with the birthday card problem, we need to adjust the level of detail to match the ability of the programmer. When in doubt, or when you are learning, it is better to have too much detail than to have too little.

Most of our examples will move from a high-level to a detailed algorithm in a single step, but this is not always reasonable. For larger, more complex problems, it is common to go through this process several times, developing intermediate level algorithms as we go. Each time, we add more detail to the previous algorithm, stopping when we see no benefit to further refinement. This technique of gradually working from a high-level to a detailed algorithm is often called stepwise refinement.

Stepwise refinement is a process for developing a detailed algorithm by gradually adding detail to a high-level algorithm.

Step 5: Review the algorithm.

The final step is to review the algorithm. What are we looking for? First, we need to work through the algorithm step by step to determine whether or not it will solve the original problem. Once we are satisfied that the algorithm does provide a solution to the problem, we start to look for other things. The following questions are typical of ones that should be asked whenever we review an algorithm. Asking these questions and seeking their answers is a good way to develop skills that can be applied to the next problem.

- Does this algorithm solve a **very specific problem** or does it solve a **more general problem**? If it solves a very specific problem, should it be generalized?

For example, an algorithm that computes the area of a circle having radius 5.2 meters (formula $\pi * 5.2^2$) solves a very specific problem, but an algorithm that computes the area of any circle (formula $\pi * R^2$) solves a more general problem.

- Can this algorithm be **simplified**?

One formula for computing the perimeter of a rectangle is:

length + width + length + width

A simpler formula would be:

$2.0 * (length + width)$



BRAINWARE UNIVERSITY

[PCC-CSM402]

[Design and Analysis of Algorithm]

- Is this solution **similar** to the solution to another problem? How are they alike? How are they different?

For example, consider the following two formulae:

Rectangle area = $length * width$

Triangle area = $0.5 * base * height$

Similarities: Each computes an area. Each multiplies two measurements.

Differences: Different measurements are used. The triangle formula contains 0.5.

Hypothesis: Perhaps every area formula involves multiplying two measurements.

Asymptotic notation

So far, we analysed linear search and binary search by counting the maximum number of guesses we need to make. But what we really want to know is *how long* these algorithms take. We're interested in *time*, not just guesses. The running times of linear search and binary search include the time needed to make and check guesses, but there's more to these algorithms.

The running time of an algorithm depends on how long it takes a computer to run the lines of code of the algorithm —and that depends on the speed of the computer, the programming language, and the compiler that translates the program from the programming language into code that runs directly on the computer, among other factors.

Let's think about the running time of an algorithm more carefully. We can use a combination of two ideas. First, we need to determine how long the algorithm takes, in terms of the size of its input. This idea makes intuitive sense, doesn't it? We've already seen that the maximum number of guesses in linear search and binary search increases as the length of the array increases. Or think about a GPS. If it knew about only the interstate highway system, and not about every little road, it should be able to find routes more quickly, right? So we think about the running time of the algorithm as a *function of the size of its input*.

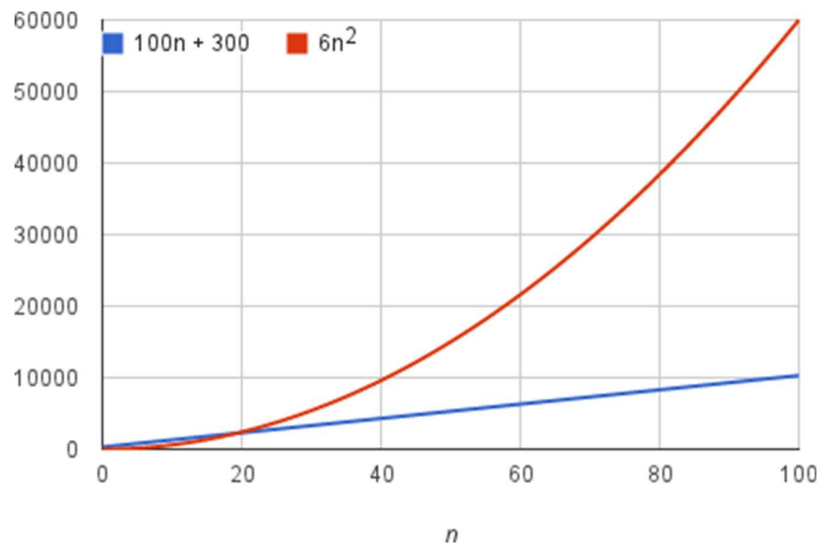
The second idea is that we must focus on how fast a function grows with the input size. We call this the **rate of growth** of the running time. To keep things manageable, we need to simplify the function to distill the most important part and cast aside the less important parts. For example, suppose that an algorithm, running on an input of size n , takes $6n^2 + 100n + 300$ machine instructions. The $6n^2$ term becomes larger than the remaining terms, $100n + 300$ once n becomes large enough, 20 in this case. Here's a chart showing values of $6n^2$ and $100n + 300$ for values of n from 0 to 100:



BRAINWARE UNIVERSITY

[PCC-CSM402]

[Design and Analysis of Algorithm]



Recurrence Relation

A **recurrence relation** is an equation which represents a sequence based on some rule. It helps in finding the subsequent term (next term) dependent upon the preceding term (previous term). If we know the previous term in a given series, then we can easily determine the next term. Since a standard pattern is developed now, we can find the set of new terms. This is also applicable for arithmetic and geometric sequence.

Recurrence Relation Definition

When we speak about a standard pattern, all the terms in the relation or equation have the same characteristics. It means if there is a value of 'n', it can be used to determine the other values by just entering the value of 'n'.

The value of n should be organised and accurate, which is known as the Simplest form. In case of the simplest form of any such relation, the next term is dependent only upon the previous term. The sequence or series generated by recurrence relation is called a **Recurrence Sequence**.

Recurrence Relation Formula

Let us assume x_n is the nth term of the series. Then the recurrence relation is shown in the form of;

$$x_{n+1} = f(x_n) ; n > 0$$

Where $f(x_n)$ is the function.

We can also define a recurrence relation as an expression that represents each element of a series as a function of the preceding ones.

$$x_n = f(n, x_{n-1}) ; n > 0$$



BRAINWARE UNIVERSITY

[PCC-CSM402]

[Design and Analysis of Algorithm]

To write the recurrence relation of first-order, say order k, the above formula can be represented as;

$$x_n = f(n, x_{n-1}, x_{n-2}, \dots, x_{n-k}) ; n > 0$$

Examples of Recurrence Relation

In Mathematics, we can see many examples of recurrence based on series and sequence pattern. Let us see some of the examples here.

Factorial Representation

We can define the factorial by using the concept of recurrence relation, such as;

$$n! = n(n-1)! ; n > 0$$

When $n = 0$,

$0! = 1$ is the initial condition.

To find the further values we have to expand the factorial notation, where the succeeding term is dependent on the preceding one.

Fibonacci Numbers

In Fibonacci numbers or series, the succeeding terms are dependent on the last two preceding terms. Therefore, this series is the best example of recurrence. As we know from the definition of the Fibonacci sequence,

$$F_n = F_{n-1} + F_{n-2}$$

Now, if we take the initial values;

$$F_0 = 0 \text{ and } F_1 = 1$$

$$\text{So, } F_2 = F_1 + F_0 = 0 + 1 = 1$$

In the same way, we can find the next succeeding terms, such as;

$$F_3 = F_2 + F_1$$

$$F_4 = F_3 + F_2$$

And so on.

Thus, the Fibonacci series is given by;

$$0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, \dots \infty$$

In the same way, there are other examples of recurrence such as a logical map, binomial coefficients where the same concept is applicable. Also, arithmetic and geometric series could be called a recurrence sequence.

Solving Recurrence Relations



BRAINWARE UNIVERSITY

[PCC-CSM402]

[Design and Analysis of Algorithm]

To solve given recurrence relations we need to find the initial term first. Suppose we have been given a sequence;

$$a_n = 2a_{n-1} - 3a_{n-2}$$

- Now the first step will be to check if initial conditions $a_0 = 1$, $a_1 = 2$, gives a closed pattern for this sequence.
- Then try with other initial conditions and find the closed formula for it.
- The result so obtained after trying different initial condition produces a series.
- Check the difference between each term, it will also form a sequence.
- We need to add all the terms of the new sequence, to understand which sequence is formed
- After understanding the pattern we can now identify the initial condition of the recurrence relation.

Recurrence Relation Problem

Now let us solve a problem based on the solution provided above.

Question: Solve the recurrence relation $a_n = a_{n-1} - n$ with the initial term $a_0 = 4$.

Solution: Let us write the sequence based on the equation given starting with the initial number.

The sequence will be 4,5,7,10,14,19,.....

Now see the difference between each term.

$$a_1 - a_0 = 1$$

$$a_2 - a_1 = 2$$

$$a_3 - a_2 = 3$$

.....

$$a_n - a_{n-1} = n$$

and so on.

Now adding all these equations both at the right-hand side, we get;

$$1 + 2 + 3 + 4 + \dots + n = 1/2 [n(n+1)]$$

Whereas on the left-hand side we get;

$$(a_1 - a_0) + (a_2 - a_1) + (a_3 - a_2) + \dots + (a_{n-1} - a_{n-2}) + (a_n - a_{n-1})$$

So you can see, all the terms get cancelled but $-a_0$ and a_n

$$\text{Therefore, } a_n - a_0 = 1/2 [n(n+1)]$$

or

$$a_n = 1/2 [n(n+1)] + a_0$$

Hence, the solution to the recurrence relation with initial condition $a_0 = 4$, is;

$$a_n = 1/2 [n(n+1)] + 4$$



BRAINWARE UNIVERSITY

[PCC-CSM402]

[Design and Analysis of Algorithm]

Master Theorem

Solve Recurrence Relation Using Master Theorem / Method

Combining the three cases above gives us the following “master theorem”.

Theorem 1 *The recurrence*

$$T(n) = aT(n/b) + cn^k$$

$$T(1) = c,$$

where a , b , c , and k are all constants, solves to:

$$T(n) \in \Theta(n^k) \text{ if } a < b^k$$

$$T(n) \in \Theta(n^k \log n) \text{ if } a = b^k$$

$$T(n) \in \Theta(n^{\log_b a}) \text{ if } a > b^k$$

To solve a recurrence relation running time you can use many different techniques. One popular technique is to use the **Master Theorem** also known as the **Master Method**. “ In the analysis of algorithms, the **master theorem** provides a solution in asymptotic terms (using Big O notation) for recurrence relations of types that occur in the analysis of many divide and conquer algorithms.

Case-01:



BRAINWARE UNIVERSITY

[PCC-CSM402]

[Design and Analysis of Algorithm]

If $a > b^k$, then $T(n) = \theta(n^{\log_b a})$

Case-02:

If $a = b^k$ and

- If $p < -1$, then $T(n) = \theta(n^{\log_b a})$
- If $p = -1$, then $T(n) = \theta(n^{\log_b a} \log^2 n)$
- If $p > -1$, then $T(n) = \theta(n^{\log_b a} \log^{p+1} n)$

Case-03:

If $a < b^k$ and

- If $p < 0$, then $T(n) = O(n^k)$
- If $p \geq 0$, then $T(n) = \theta(n^k \log^p n)$

PRACTICE PROBLEMS BASED ON MASTER THEOREM-

Problem-01:

Solve the following recurrence relation using Master's theorem-

$$T(n) = 3T(n/2) + n^2$$

Solution-

We compare the given recurrence relation with $T(n) = aT(n/b) + \theta(n^k \log^p n)$. Then, we have-

$$a = 3$$

$$b = 2$$

$$k = 2$$

$$p = 0$$



BRAINWARE UNIVERSITY

[PCC-CSM402]

[Design and Analysis of Algorithm]

Now, $a = 3$ and $b^k = 2^2 = 4$.

Clearly, $a < b^k$.

So, we follow case-03.

Since $p = 0$, so we have-

$$T(n) = \theta(n^k \log^p n)$$

$$T(n) = \theta(n^2 \log^0 n)$$

Thus,

$$T(n) = \theta(n^2)$$

Problem-02:

Solve the following recurrence relation using Master's theorem-

$$T(n) = 2T(n/2) + n \log n$$

Solution-

We compare the given recurrence relation with $T(n) = aT(n/b) + \theta(n^k \log^p n)$. Then, we have-

$$a = 2$$

$$b = 2$$

$$k = 1$$

$$p = 1$$

Now, $a = 2$ and $b^k = 2^1 = 2$.



BRAINWARE UNIVERSITY

[PCC-CSM402]

[Design and Analysis of Algorithm]

Clearly, $a = b^k$.

So, we follow case-02.

Since $p = 1$, so we have-

$$\begin{aligned} T(n) &= \theta(n^{\log_b \log^{p+1} n}) T(n) \\ &= \theta(n^{\log_2 \log^{1+1} n}) \end{aligned}$$

Thus,

$$T(n) = \theta(n \log^2 n)$$

Problem-03:

Solve the following recurrence relation using Master's theorem-

$$T(n) = 2T(n/4) + n^{0.51}$$

Solution-

We compare the given recurrence relation with $T(n) = aT(n/b) + \theta(n^k \log^p n)$. Then, we have-

$$a = 2$$

$$b = 4$$

$$k = 0.51$$

$$p = 0$$

Now, $a = 2$ and $b^k = 4^{0.51} = 2.0279$.

Clearly, $a < b^k$.

So, we follow case-03.



BRAINWARE UNIVERSITY

[PCC-CSM402]

[Design and Analysis of Algorithm]

Since $p = 0$, so we have-

$$T(n) = \theta(n^k \log^p n)$$

$$T(n) = \theta(n^{0.51} \log^0 n)$$

Thus,

$$T(n) = \theta(n^{0.51})$$

Problem-04:

Solve the following recurrence relation using Master's theorem-

$$T(n) = \sqrt{2}T(n/2) + \log n$$

Solution-

We compare the given recurrence relation with $T(n) = aT(n/b) + \theta(n^k \log^p n)$. Then, we have-

$$a = \sqrt{2}$$

$$b = 2$$

$$k = 0$$

$$p = 1$$

Now, $a = \sqrt{2} = 1.414$ and $b^k = 2^0 = 1$.

Clearly, $a > b^k$.

So, we follow case-01.

So, we have-



BRAINWARE UNIVERSITY

[PCC-CSM402]

[Design and Analysis of Algorithm]

$$T(n) = \theta (n^{\log_b a})$$

$$T(n) = \theta (n^{\log_2 \sqrt{2}})$$

$$T(n) = \theta (n^{1/2})$$

Thus,

$$T(n) = \theta (\sqrt{n})$$

Problem-05:

Solve the following recurrence relation using Master's theorem-

$$T(n) = 8T(n/4) - n^2 \log n$$

Solution-

- The given recurrence relation does not correspond to the general form of Master's theorem.
- So, it can not be solved using Master's theorem.

Problem-06:

Solve the following recurrence relation using Master's theorem-

$$T(n) = 3T(n/3) + n/2$$

Solution-

- We write the given recurrence relation as $T(n) = 3T(n/3) + n$.
- This is because in the general form, we have θ for function $f(n)$ which hides constants in it.
- Now, we can easily apply Master's theorem.



BRAINWARE UNIVERSITY

[PCC-CSM402]

[Design and Analysis of Algorithm]

We compare the given recurrence relation with $T(n) = aT(n/b) + \theta(n^k \log^p n)$. Then, we have-

$$a = 3$$

$$b = 3$$

$$k = 1$$

$$p = 0$$

Now, $a = 3$ and $b^k = 3^1 = 3$.

Clearly, $a = b^k$.

So, we follow case-02.

Since $p = 0$, so we have-

$$T(n) = \theta(n^{\log_b a} \cdot \log^{p+1} n)$$

$$T(n) = \theta(n^{\log_3 3} \cdot \log^{0+1} n)$$

$$T(n) = \theta(n^1 \cdot \log^1 n)$$

Thus,

$$T(n) = \theta(n \log n)$$

Problem-07:

Form a recurrence relation for the following code and solve it using Master's theorem-

```
A(n)
{
  if(n<=1)
  return 1;
  else
```



BRAINWARE UNIVERSITY

[PCC-CSM402]

[Design and Analysis of Algorithm]

```
return A( $\sqrt{n}$ );  
}
```

Solution-

- We write a recurrence relation for the given code as $T(n) = T(\sqrt{n}) + 1$.
- Here 1 = Constant time taken for comparing and returning the value.
- We can not directly apply Master's Theorem on this recurrence relation.
- This is because it does not correspond to the general form of Master's theorem.
- However, we can modify and bring it in the general form to apply Master's theorem.

Let-

$$n = 2^m \dots\dots(1)$$

Then-

$$T(2^m) = T(2^{m/2}) + 1$$

Now, let $T(2^m) = S(m)$, then $T(2^{m/2}) = S(m/2)$

So, we have-

$$S(m) = S(m/2) + 1$$

Now, we can easily apply Master's Theorem.

We compare the given recurrence relation with $S(m) = aS(m/b) + \theta(m^k \log^p m)$.

Then, we have-

$$a = 1$$

$$b = 2$$

$$k = 0$$

$$p = 0$$

Now, $a = 1$ and $b^k = 2^0 = 1$.



BRAINWARE UNIVERSITY

[PCC-CSM402]

[Design and Analysis of Algorithm]

Clearly, $a = b^k$.

So, we follow case-02.

Since $p = 0$, so we have-

$$S(m) = \theta(m^{\log_b a} \cdot \log^{p+1} m)$$

$$S(m) = \theta(m^{\log_2 1} \cdot \log^{0+1} m)$$

$$S(m) = \theta(m^0 \cdot \log^1 m)$$

Thus,

$$S(m) = \theta(\log m) \dots\dots(2)$$

Now,

- From (1), we have $n = 2^m$.
- So, $\log n = m \log 2$ which implies $m = \log_2 n$.

Substituting in (2), we get-

$$S(m) = \theta(\log \log_2 n)$$

$$T(n) = \theta(\log \log_2 n)$$

Substitution method for solving recurrences

Introduction

Recurrence relations are equations that describe themselves. We encounter recurrences in various situations when we have to obtain the asymptotic bound on the number of $O(1)$ operations (constant time operations, ones that aren't affected by the size of the input) performed by that recursive function. It's essential to have tools to solve these



BRAINWARE UNIVERSITY

[PCC-CSM402]

[Design and Analysis of Algorithm]

recurrences for time complexity analysis, and here the substitution method comes into the picture.

Substitution Method

In the substitution method, we have a known recurrence, and we use induction to prove that our guess is a good bound for the recurrence's solution. This method works well in providing us with a good upper bound in most recurrences that can't be solved using the Master's Theorem or other more straightforward ways.

Practising similar questions is essential if we try to make a guess that'll probably work for us. We'll cover some examples to get you started in the next section.

The steps to use the Substitution method are as follows.

1. Guess a solution through your experience.
2. Use induction to prove that the guess is an upper bound solution for the given recurrence relation.

Examples of the process of solving recurrences using substitution

Let's say we have the recurrence relation given below.

$$T(n) = 2 * T(n-1) + c1, (n > 1)$$
$$T(1) = 1$$

We know that the answer is probably $T(N) = O(2n)$. The observation that we are almost doubling the number of $O(1)$ operations for a constant decrease in n leads to the guess. We will thus show that $T(n) \leq k * 2n - b$ and prove our answer.

Now we come over to the use of induction, here the base case occurs when $n=1$.

$$\Rightarrow T(1) = 1 \leq k * 21 - b = 2 * k - b.$$
$$\Rightarrow 2 * k - b \geq 1$$
$$\Rightarrow k \geq (b+1)/2$$

Inductive step: We assume that the property that we guessed holds for $n - 1$ and prove that in such a case it will also hold for n .

$$\Rightarrow T(n) = 2 * (T(n-1)) + c1$$
$$\Rightarrow T(n) \leq 2 * (k * 2n-1 - b) + c1$$
$$\Rightarrow T(n) \leq k * 2n - 2 * b + c1$$
$$\Rightarrow T(n) \leq k * 2n - b \text{ (let, } b = c1)$$



BRAINWARE UNIVERSITY

[PCC-CSM402]

[Design and Analysis of Algorithm]

After the above steps we obtain, $b = c_1$ and $k = (b + 1) / 2$. These equations have infinite solutions, and we are free to choose any constant as big-O notation doesn't care about constants being multiplied, divided, added or subtracted. Hence, we have confirmed that $T(n) = O(2n)$.

Let's cover another example. The recurrence relation is described below.

$$T(n) = 2 * T(n / 2) + n, n > 1$$

$$T(1) = 1$$

$T(n) = O(n \log(n))$ is our guess. We must therefore show that $T(n) \leq c_1 n \log(n) + c_2$ to prove our guess.

Base case ($n = 1$)

$$\Rightarrow T(1) = 1 \leq c_1 * 1 * \log(1) + c_2$$

$$\Rightarrow 1 \leq c_1 * 1 * 0 + c_2$$

$$\Rightarrow c_2 \geq 1$$

Inductive step, ($n > 1$)

$$\Rightarrow T(n) \leq 2 * T(n / 2) + n$$

$$\Rightarrow T(n) \leq 2 * (c_1 * n / 2 * \log(n / 2) + c_2) + n$$

$$\Rightarrow T(n) \leq c_1 * n * \log(n) - c_1 * n * \log(2) + 2 * c_2 + n$$

$$\Rightarrow T(n) \leq c_1 * n * \log(n) + (1 - c_1 * \log(2)) * n + 2 * c_2$$

We can conclude that the above holds for some $c_1 \geq 1 / \log(2)$ and some c_2 . We again have numerous solutions for the above set of equations, and hence the guess is accurate and $T(n) = O(n * \log(n))$.

FAQs

1. What are recurrences and their uses?

A recurrence can describe any sequence of elements in an array in terms of themselves. Recurrences are used in various branches of mathematics in addition to Computer Science (in time complexity analysis) their application in finding the bound for the number of trivial operations by an algorithm, such as Number Theory (Fibonacci sequence), Calculus (Euler's Theorem), etc.

An example of a recurrence,

$$T(N) = T(N/2) + 1, (n > 1)$$

$$T(1) = 1$$

2. Why should we learn about different methods for solving recurrences?

Many methods make it easier for us to solve particular problems. Depending on the requirement, they can also help us (depending on if we need a tight bound or just an upper bound), and there are recurrences where you may not apply specific methods.

For example, Master's Theorem can't solve recurrences if in the recurrence $T(n) = a * T(n / b) + f(n)$, $f(n)$ is not a polynomial.

3. What is induction and its uses?

It's a proving method, generally used to prove that a statement is true for all-natural numbers. The



BRAINWARE UNIVERSITY

[PCC-CSM402]

[Design and Analysis of Algorithm]

key idea is to establish the base case and then prove that if a statement is true, the next one also has to be true. It's used as a tool to prove various statements in mathematics and computer science, such as the sum of all-natural numbers $S(n) = (n * (n + 1)) / 2$.

4. What is Master's Theorem?

It's another method to solve recurrences. It covers many common recurrences that generally appear in many "divide and conquer" algorithms and provides a complete and straightforward way for solving such recurrences. Its usage, however, has limitations and can't always be used.

5. Where we shouldn't use the Substitution Method?

Let's say we have recurrences that satisfies all the requirements for master's theorem. Then it's best to use it. Methods that provide a definite answer easily and don't require guessing are usually faster and need much less work to give even a tight bound for our recurrence using master's theorem, unlike the substitution method.

Extra Methods:

Using the substitution and master methods

Using the substitution method

The substitution method is a condensed way of proving an asymptotic bound on a recurrence by induction. In the substitution method, instead of trying to find an exact closed-form solution, we only try to find a closed-form *bound* on the recurrence. This is often much easier than finding a full closed-form solution, as there is much greater leeway in dealing with constants.

The substitution method is a powerful approach that is able to prove upper bounds for almost all recurrences. However, its power is not always needed; for certain types of recurrences, the master method (see below) can be used to derive a tight bound with less work. In those cases, it is better to simply use the master method, and to save the substitution method for recurrences that actually need its full power.

Note that the substitution method still requires the use of induction. The induction will always be of the same basic form, but it is still important to state the property you are trying to prove, split into one or more base cases and the inductive case, and note when the inductive hypothesis is being used.

Substitution method example

Consider the following recurrence relation, which shows up fairly frequently for some types of algorithms:

$$T(1) = 1$$

$$T(n) = 2T(n-1) + c_1$$

By expanding this out a bit (using the "iteration method"), we can guess that this will be $O(2^n)$. To use the substitution method to prove this bound, we now need to guess a closed-form upper bound based on this asymptotic bound. We will guess an upper bound



BRAINWARE UNIVERSITY

[PCC-CSM402]

[Design and Analysis of Algorithm]

of $k2^n - b$, where b is some constant. We include the b in anticipation of having to deal with the constant c_1 that appears in the recurrence relation, and because it does no harm. In the process of proving this bound by induction, we will generate a set of constraints on k and b , and if b turns out to be unnecessary, we will be able to set it to whatever we want at the end.

Our property, then, is $T(n) \leq k2^n - b$, for some two constants k and b . Note that this property logically implies that $T(n)$ is $O(2^n)$, which can be verified with reference to the definition of O .

Base case: $n = 1$. $T(1) = 1 \leq k2^1 - b = 2k - b$. This is true as long as $k \geq (b + 1)/2$.

Inductive case: We assume our property is true for $n - 1$. We now want to show that it is true for n .

$$\begin{aligned} T(n) &= 2T(n-1) + c_1 \\ &\leq 2(k2^{n-1} - b) + c_1 \quad (\text{by IH}) \\ &= k2^n - 2b + c_1 \\ &\leq k2^n - b \end{aligned}$$

This is true as long as $b \geq c_1$.

So we end up with two constraints that need to be satisfied for this proof to work, and we can satisfy them simply by letting $b = c_1$ and $k = (b + 1)/2$, which is always possible, as the definition of O allows us to choose any constant. Therefore, we have proved that our property is true, and so $T(n)$ is $O(2^n)$.

The biggest thing worth noting about this proof is the importance of adding additional terms to the upper bound we assume. In almost all cases in which the recurrence has constants or lower-order terms, it will be necessary to have additional terms in the upper bound to "cancel out" the constants or lower-order terms. Without the right additional terms, the inductive case of the proof will get stuck in the middle, or generate an impossible constraint; this is a signal to go back to your upper bound and determine what else needs to be added to it that will allow the proof to proceed without causing the bound to change in asymptotic terms.

Review of the master method

The master method gives us a quick way to find solutions to recurrence relations of the form $T(n) = aT(n/b) + h(n)$, where a and b are constants, $a \geq 1$ and $b > 1$. Conceptually, a represents how many recursive calls are made, b represents the factor by which the work is reduced in each recursive call, and $h(n)$ represents how much work is done by each call apart from the recursion, as a function of n .

Once we have a recurrence relation of that form, the master method tells us the solution based on the relation between a , b , and $h(n)$, as follows:



BRAINWARE UNIVERSITY

[PCC-CSM402]

[Design and Analysis of Algorithm]

- Case 1: $h(n)$ is $O(n^{\log_b a - \epsilon})$, which says that h grows more slowly than the number of leaves. In this case, the total work is dominated by the work done at the leaves, so $T(n)$ is $\Theta(n^{\log_b a})$.
- Case 2: $h(n)$ is $\Theta(n^{\log_b a})$, which says that h grows at the same rate as the number of leaves. In this case, $T(n)$ is $\Theta(n^{\log_b a} \log n)$.
- Case 3: $h(n)$ is $\Omega(n^{\log_b a + \epsilon})$, which says that h grows faster than the number of leaves. For the upper bound, we also need an extra smoothness condition on f , namely $ah(n/b) \leq ch(n)$ for some $c < 1$ and large n . In this case, the total work is dominated by the work done at the root, so $T(n)$ is $\Theta(h(n))$.

Using the master method for single recurrences

The simplest application of the master method is to a recurrence relation with fixed a , b , and $h(n)$. Given such a recurrence relation, all you need to do is prove it falls under a particular case and then present the conclusion derived from that case. Note that the master method does not help you come up with a recurrence relation in the first place; all it does is help you solve recurrence relations once you have them. Therefore, for analyzing the runtime of algorithms, the first step still must be to derive a recurrence relation for the runtime.

Examples for the master method

Example 1: Say you have derived the recurrence relation $T(n) = 8T(n/2) + cn^2$, where c is some positive constant. We see that this has the appropriate form for applying the master method, and that $a=8$, $b=2$, and $h(n) = cn^2$. cn^2 is $O(n^{\log_2 8 - \epsilon}) = O(n^{3 - \epsilon})$ for any $\epsilon \leq 1$, so this falls into case 1. Therefore, $T(n)$ is $\Theta(n^3)$.

Example 2: Say you have derived the recurrence relation $T(n) = T(n/2) + cn$, where c is some positive constant. We see that this has the appropriate form for applying the master method, and that $a=1$, $b=2$, and $h(n) = cn$. Then $h(n)$ is $\Omega(n^{\log_2 1 + \epsilon}) = \Omega(n^\epsilon)$ for any $\epsilon \leq 1$, so this falls into case 3. And $ah(n/b) = cn/2 = \frac{1}{2}h(n)$, therefore $T(n)$ is $\Theta(n)$.

Example 3: Say you have derived the recurrence relation $T(n) = 8T(n/4) + cn^{3/2}$, where c is some positive constant. We see that this has the appropriate form for applying the master method, and that $a=8$, $b=4$, and $h(n) = cn^{3/2}$. $cn^{3/2}$ is $\Theta(n^{\log_4 8}) = \Theta(n^{3/2})$, so this falls into case 2. Therefore, $T(n)$ is $\Theta(n^{3/2} \log n)$.

Using the master method for classes of recurrences

The master method can also be used to analyze recurrences where one of a , b , or $h(n)$ varies. The structure of the master method's three cases in asymptotic is roughly that of a "less than" case, an "equal" case, and a "greater than" case, and so given a recurrence relation with a , b , or $h(n)$ unknown, we can use the master method to analyze it for a range of values of the unknown by determining the value of the unknown at which the recurrence relation falls into case 2 (the "equal") case. Once that is shown, it is usually



BRAINWARE UNIVERSITY

[PCC-CSM402]

[Design and Analysis of Algorithm]

easy to prove that all greater values of the unknown will be covered by one of the two remaining cases, and all lesser values of the unknown will be covered by the final case.

As a and b , the range for them will typically be all values for which the master method can be applied. For $h(n)$, it is impossible to analyze for all possible functions, so the range will normally be restricted to functions of a particular type, such as cn^k for $k \geq 0$.

Consider, for example, the recurrence relation $T(n) = 6T(n/b) + cn^2$, with $a=6$, $h(n) = cn^2$, and b and c unknown. To fall into case 2, we need cn^2 to be $\Theta(n^{\log_b 6})$, which requires $\log_b 6 = 2$, so $b = \sqrt{6}$. **And we can see that for $b > \sqrt{6}$, we will have $\log_b 6 < 2$, so we will fall into case 3, and for $b < \sqrt{6}$, we will have $\log_b 6 > 2$, so we will fall into case 1.**

Applying those three cases, we determine that $T(n)$ is:

- $\Theta(n^{\log_b 6})$ for $b < \sqrt{6}$
- $\Theta(n^{\log_b 6} \log n)$ for $b = \sqrt{6}$
- $\Theta(n^2)$ for $b > \sqrt{6}$, because $6h(n/b) = 6c(n/b)^2 = (6/b^2)(cn^2) = (6/b^2)h(n)$ and $6/b^2 < 1$.

Note that as occurred in the example, analyzing a class of recurrence relations in this fashion may produce a "tipping point" for the case-2 value that is not an integer (and might not make sense in the original context). This is not worrisome — even if the original context is integer-only, we still learn where the split between case 1 and case 3 occurs.

Recurrence relations not solvable by the master method

While the master method is very useful in practice, it is worth keeping in mind that there are recurrence relations that it cannot solve. It is always important to verify that the conditions required by the master method hold, as it is possible for a recurrence relation to superficially look to be in the right form, but still violate the requirements or not fall into any of the cases.

For example, $T(n) = 2T(n/2) + n/(\log n)$ satisfies all the explicit requirements: we have $a=2$, $b=2$, and $h(n) = n/(\log n)$. Nevertheless, it does not fit any of the cases:

- Case 1: $n/(\log n)$ is not $O(n^{1-\epsilon})$ for any $\epsilon > 0$, since n^ϵ grows faster than $\log n$
- Case 2: $n/(\log n)$ is not $\Theta(n)$, since $(n)/(n/(\log n)) = \log n$
- Case 3: $n/(\log n)$ is not $\Omega(n^{1+\epsilon})$ for any $\epsilon > 0$, since $(n^{1+\epsilon})/(n/(\log n)) = n^\epsilon \log n$

So we actually can't use the master method to solve this recurrence relation. We can, however, still derive an upper bound for this recurrence by using a little trick: we find a similar recurrence that is larger than $T(n)$, analyze the new recurrence using the master method, and use the result as an upper bound for $T(n)$.



BRAINWARE UNIVERSITY

[PCC-CSM402]

[Design and Analysis of Algorithm]

$T(n) = 2T(n/2) + n/(\log n) \leq 2T(n/2) + n$, so if we call $R(n)$ the function such that $R(n) = 2T(n/2) + n$, we know that $R(n) \geq T(n)$. This is something we can apply the master method to: n is $\Theta(n)$, so $R(n)$ is $\Theta(n \log n)$. Since $T(n) \leq R(n)$, we can conclude that $T(n)$ is $O(n \log n)$. Note that we only end up with O , not Θ ; since we were only able to apply the master method indirectly, we could not show a tight bound.

Recursion Tree Method

1. Recursion Tree Method is a pictorial representation of an iteration method which is in the form of a tree where at each level nodes are expanded.
2. In general, we consider the second term in recurrence as root.
3. It is useful when the divide & Conquer algorithm is used.
4. It is sometimes difficult to come up with a good guess. In Recursion tree, each root and child represents the cost of a single subproblem.
5. We sum the costs within each of the levels of the tree to obtain a set of pre-level costs and then sum all pre-level costs to determine the total cost of all levels of the recursion.
6. A Recursion Tree is best used to generate a good guess, which can be verified by the Substitution Method.

Example 1

$$\text{Consider } T(n) = 2T\left(\frac{n}{2}\right) + n^2$$

We have to obtain the asymptotic bound using recursion tree method.

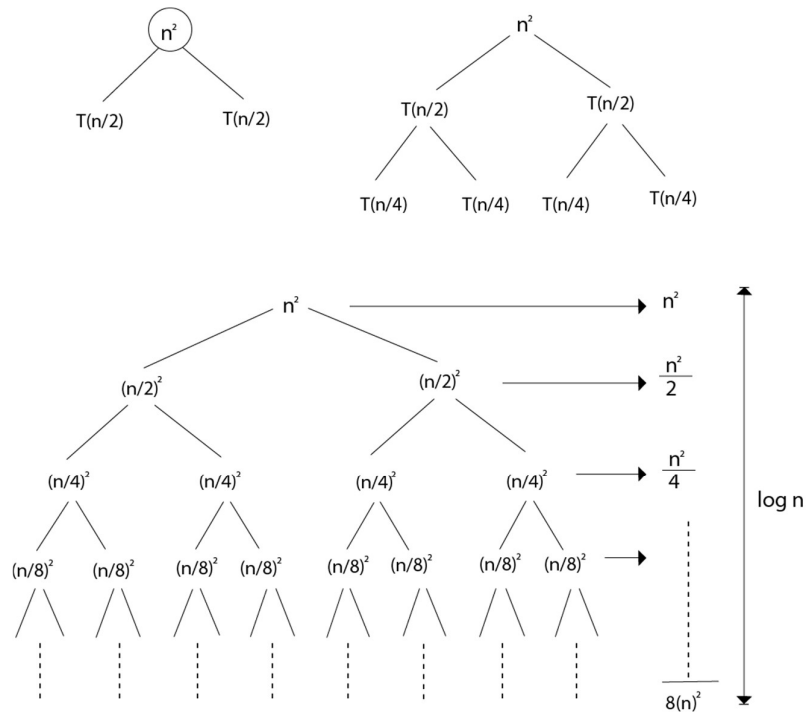
Solution: The Recursion tree for the above recurrence is



BRAINWARE UNIVERSITY

[PCC-CSM402]

[Design and Analysis of Algorithm]



$$T(n) = n^2 + \frac{n^2}{2} + \frac{n^2}{4} + \dots \log n \text{ times.}$$

$$\leq n^2 \sum_{i=0}^{\infty} \left(\frac{1}{2^i} \right)$$

$$\leq n^2 \left(\frac{1}{1 - \frac{1}{2}} \right) \leq 2n^2$$

$$T(n) = \Theta(n^2)$$

Example 2: Consider the following recurrence

$$T(n) = 4T\left(\frac{n}{2}\right) + n$$

Obtain the asymptotic bound using recursion tree method.

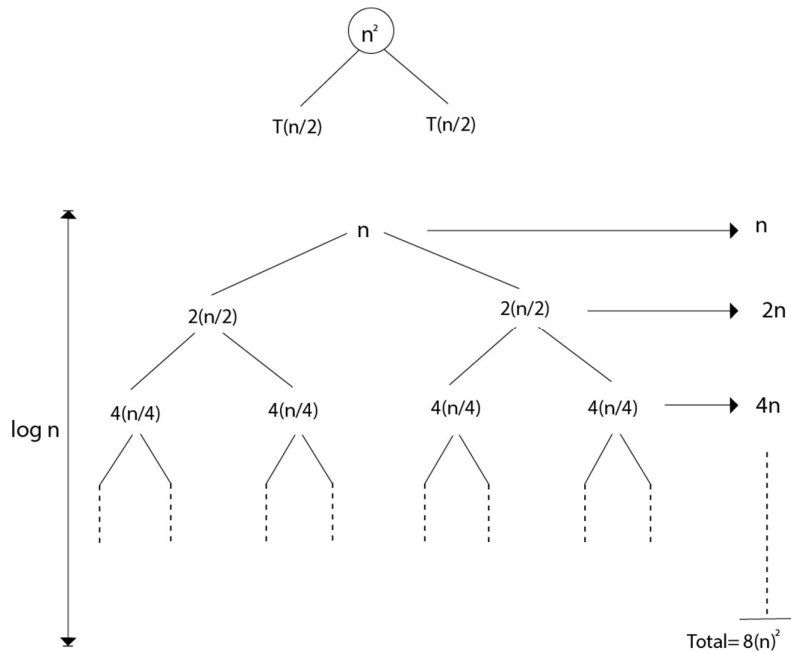
Solution: The recursion trees for the above recurrence



BRAINWARE UNIVERSITY

[PCC-CSM402]

[Design and Analysis of Algorithm]



We have $n + 2n + 4n + \dots \log_2 n$ times

$$= n (1 + 2 + 4 + \dots \log_2 n \text{ times})$$

$$= n \frac{(2 \log_2 n - 1)}{(2 - 1)} = \frac{n(n-1)}{1} = n^2 - n = \theta(n^2)$$

$$T(n) = \theta(n^2)$$

Example 3: Consider the following recurrence

$$T(n) = T\left(\frac{n}{3}\right) + T\left(\frac{2n}{3}\right) + n$$

Obtain the asymptotic bound using recursion tree method.

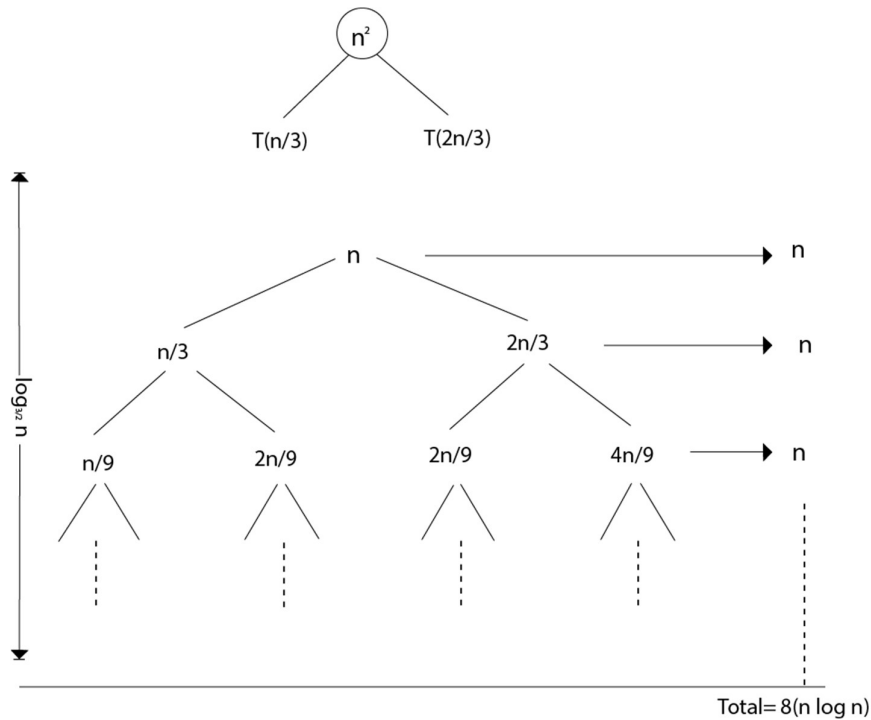
Solution: The given Recurrence has the following recursion tree



BRAINWARE UNIVERSITY

[PCC-CSM402]

[Design and Analysis of Algorithm]



When we add the values across the levels of the recursion trees, we get a value of n for every level. The longest path from the root to leaf is

$$n \rightarrow \frac{2}{3}n \rightarrow \left(\frac{2}{3}\right)^2 n \rightarrow \dots 1$$

Since $\left(\frac{2}{3}\right)^i n = 1$ when $i = \log_{3/2} n$.

Thus the height of the tree is $\log_{3/2} n$.

$$T(n) = n + n + n + \dots + \log_{3/2} n \text{ times.} = \theta(n \log n)$$

Steps to Solve Recurrence Relations Using Recursion Tree Method-

Step-01:



BRAINWARE UNIVERSITY

[PCC-CSM402]

[Design and Analysis of Algorithm]

Draw a recursion tree based on the given recurrence relation.

Step-02:

Determine-

- Cost of each level
- Total number of levels in the recursion tree
- Number of nodes in the last level
- Cost of the last level

Step-03:

Add cost of all the levels of the recursion tree and simplify the expression so obtained in terms of asymptotic notation.

Following problems clearly illustrates how to apply these steps.

PRACTICE PROBLEMS BASED ON RECURSION TREE-

Problem-01:

Solve the following recurrence relation using recursion tree method-

$$T(n) = 2T(n/2) + n$$

Solution-

Step-01:



BRAINWARE UNIVERSITY

[PCC-CSM402]

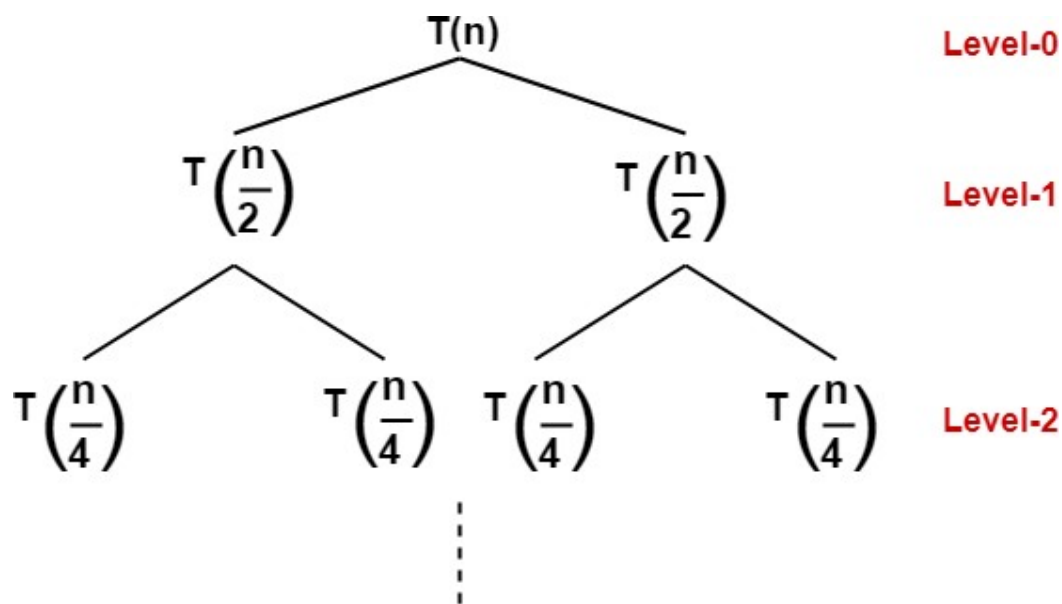
[Design and Analysis of Algorithm]

Draw a recursion tree based on the given recurrence relation.

The given recurrence relation shows-

- A problem of size n will get divided into 2 sub-problems of size $n/2$.
- Then, each sub-problem of size $n/2$ will get divided into 2 sub-problems of size $n/4$ and so on.
- At the bottom most layer, the size of sub-problems will reduce to 1.

This is illustrated through following recursion tree-



The given recurrence relation shows-

- The cost of dividing a problem of size n into its 2 sub-problems and then combining its solution is n .
- The cost of dividing a problem of size $n/2$ into its 2 sub-problems and then combining its solution is $n/2$ and so on.

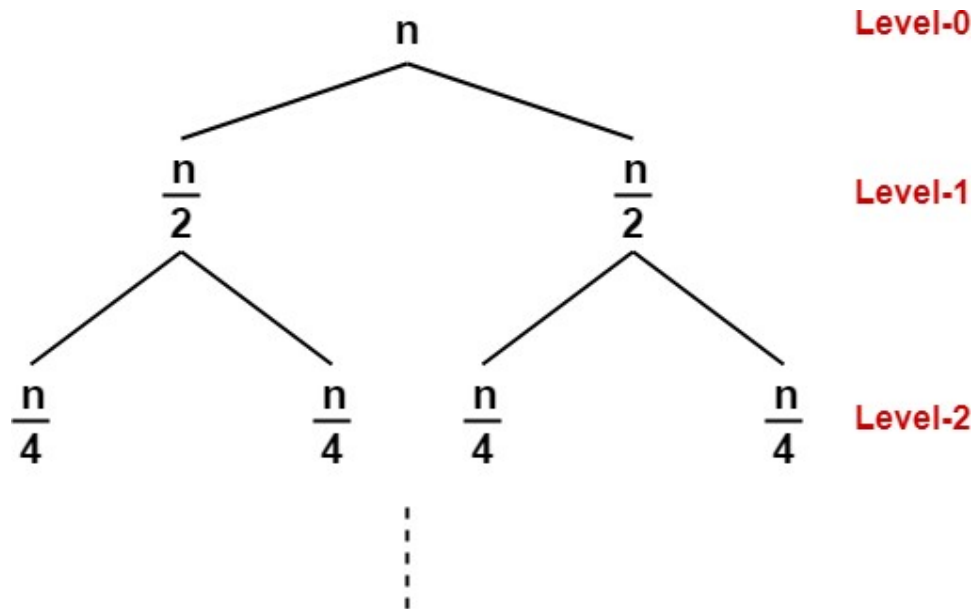
This is illustrated through following recursion tree where each node represents the cost of the corresponding sub-problem-



BRAINWARE UNIVERSITY

[PCC-CSM402]

[Design and Analysis of Algorithm]



Step-02:

Determine cost of each level-

- Cost of level-0 = n
- Cost of level-1 = $n/2 + n/2 = n$
- Cost of level-2 = $n/4 + n/4 + n/4 + n/4 = n$ and so on.

Step-03:

Determine total number of levels in the recursion tree-

- Size of sub-problem at level-0 = $n/2^0$
- Size of sub-problem at level-1 = $n/2^1$
- Size of sub-problem at level-2 = $n/2^2$

Continuing in similar manner, we have-

$$\text{Size of sub-problem at level-}i = n/2^i$$



BRAINWARE UNIVERSITY

[PCC-CSM402]

[Design and Analysis of Algorithm]

Suppose at level-x (last level), size of sub-problem becomes 1. Then-

$$n / 2^x = 1$$

$$2^x = n$$

Taking log on both sides, we get-

$$x \log 2 = \log n$$

$$x = \log_2 n$$

$\therefore$ Total number of levels in the recursion tree = $\log_2 n + 1$

Step-04:

Determine number of nodes in the last level-

- Level-0 has 2^0 nodes i.e. 1 node
- Level-1 has 2^1 nodes i.e. 2 nodes
- Level-2 has 2^2 nodes i.e. 4 nodes

Continuing in similar manner, we have-

Level- $\log_2 n$ has $2^{\log_2 n}$ nodes i.e. n nodes

Step-05:

Determine cost of last level-

$$\text{Cost of last level} = n \times T(1) = \theta(n)$$

Step-06:

Add costs of all the levels of the recursion tree and simplify the expression so obtained in terms of asymptotic notation-



BRAINWARE UNIVERSITY

[PCC-CSM402]

[Design and Analysis of Algorithm]

$$T(n) = \underbrace{\{ n + n + n + \dots \}}_{\text{For } \log_2 n \text{ levels}} + \theta(n)$$

$$= n \times \log_2 n + \theta(n)$$

$$= n \log_2 n + \theta(n)$$

$$= \theta(n \log_2 n)$$

Problem-02:

Solve the following recurrence relation using recursion tree method-

$$T(n) = T(n/5) + T(4n/5) + n$$

Solution-

Step-01:

Draw a recursion tree based on the given recurrence relation.

The given recurrence relation shows-

- A problem of size n will get divided into 2 sub-problems- one of size $n/5$ and another of size $4n/5$.
- Then, sub-problem of size $n/5$ will get divided into 2 sub-problems- one of size $n/5^2$ and another of size $4n/5^2$.
- On the other side, sub-problem of size $4n/5$ will get divided into 2 sub-problems- one of size $4n/5^2$ and another of size $4^2n/5^2$ and so on.
- At the bottom most layer, the size of sub-problems will reduce to 1.

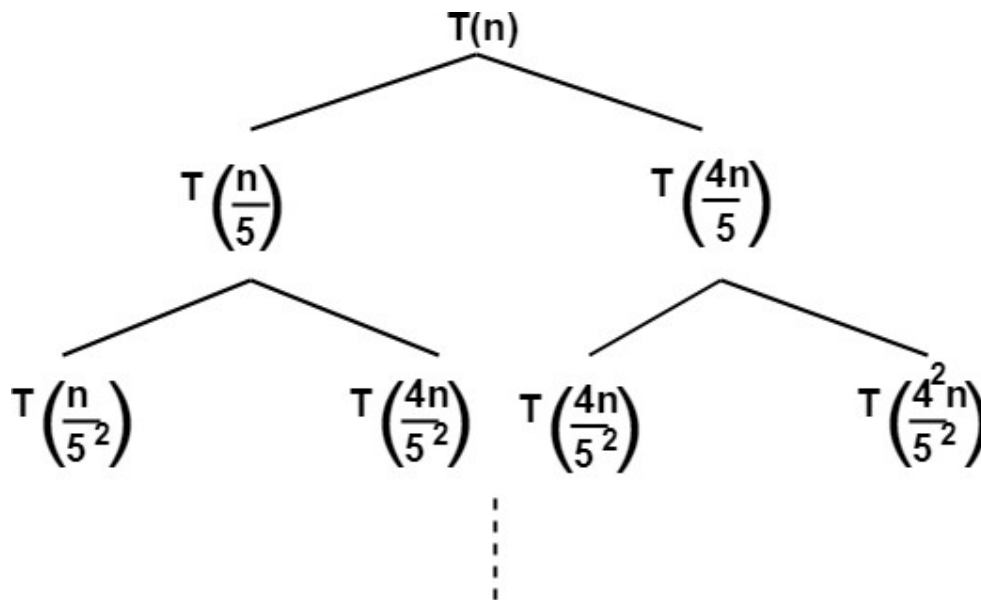
This is illustrated through following recursion tree-



BRAINWARE UNIVERSITY

[PCC-CSM402]

[Design and Analysis of Algorithm]



The given recurrence relation shows-

- The cost of dividing a problem of size n into its 2 sub-problems and then combining its solution is n .
- The cost of dividing a problem of size $n/5$ into its 2 sub-problems and then combining its solution is $n/5$.
- The cost of dividing a problem of size $4n/5$ into its 2 sub-problems and then combining its solution is $4n/5$ and so on.

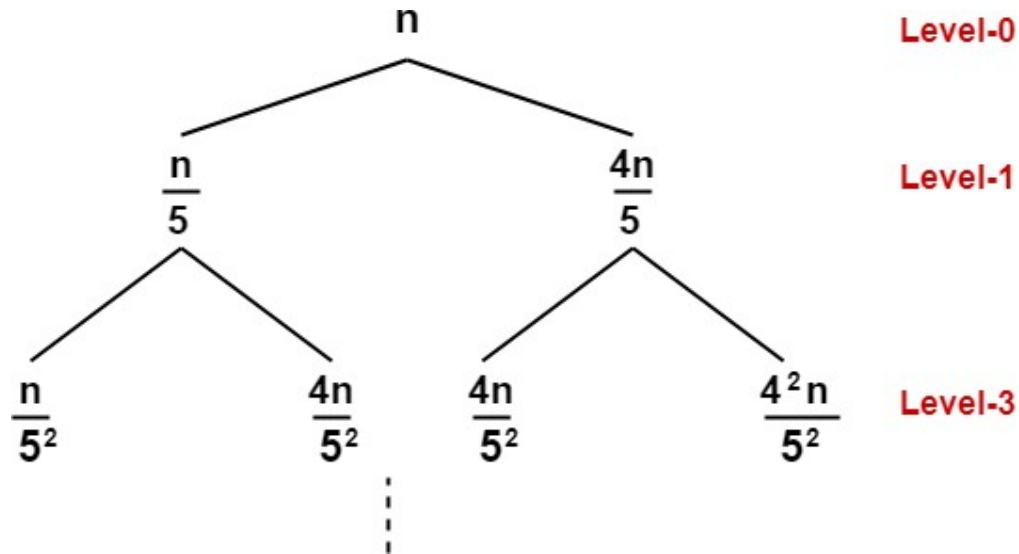
This is illustrated through following recursion tree where each node represents the cost of the corresponding sub-problem-



BRAINWARE UNIVERSITY

[PCC-CSM402]

[Design and Analysis of Algorithm]



Step-02:

Determine cost of each level-

- Cost of level-0 = n
- Cost of level-1 = $\frac{n}{5} + \frac{4n}{5} = n$
- Cost of level-2 = $\frac{n}{5^2} + \frac{4n}{5^2} + \frac{4n}{5^2} + \frac{4^2n}{5^2} = n$

Step-03:

Determine total number of levels in the recursion tree. We will consider the rightmost sub tree as it goes down to the deepest level-

- Size of sub-problem at level-0 = $(\frac{4}{5})^0n$
- Size of sub-problem at level-1 = $(\frac{4}{5})^1n$
- Size of sub-problem at level-2 = $(\frac{4}{5})^2n$

Continuing in similar manner, we have-

$$\text{Size of sub-problem at level-}i = (\frac{4}{5})^in$$

Suppose at level- x (last level), size of sub-problem becomes 1. Then-

Prepared by: Department of CSE-AI, Brainware University, Barasat



BRAINWARE UNIVERSITY

[PCC-CSM402]

[Design and Analysis of Algorithm]

$$(4/5)^x = 1$$

$$(4/5)^x = 1/n$$

Taking log on both sides, we get-

$$x \log(4/5) = \log(1/n)$$

$$x = \log_{5/4} n$$

$\therefore$ Total number of levels in the recursion tree = $\log_{5/4} n + 1$

Step-04:

Determine number of nodes in the last level-

- Level-0 has 2^0 nodes i.e. 1 node
- Level-1 has 2^1 nodes i.e. 2 nodes
- Level-2 has 2^2 nodes i.e. 4 nodes

Continuing in similar manner, we have-

Level- $\log_{5/4} n$ has $2^{\log_{5/4} n}$ nodes

Step-05:

Determine cost of last level-

$$\text{Cost of last level} = 2^{\log_{5/4} n} \times T(1) = \theta(2^{\log_{5/4} n}) = \theta(n^{\log_{5/4} 2})$$

Step-06:

Add costs of all the levels of the recursion tree and simplify the expression so obtained in terms of asymptotic notation-



BRAINWARE UNIVERSITY

[PCC-CSM402]

[Design and Analysis of Algorithm]

$$T(n) = \underbrace{\{ n + n + n + \dots \}}_{\text{For } \log_{5/4} n \text{ levels}} + \theta(n^{\log_{5/4} 2})$$

$$= n \log_{5/4} n + \theta(n^{\log_{5/4} 2})$$

$$= \theta(n \log_{5/4} n)$$

Problem-03:

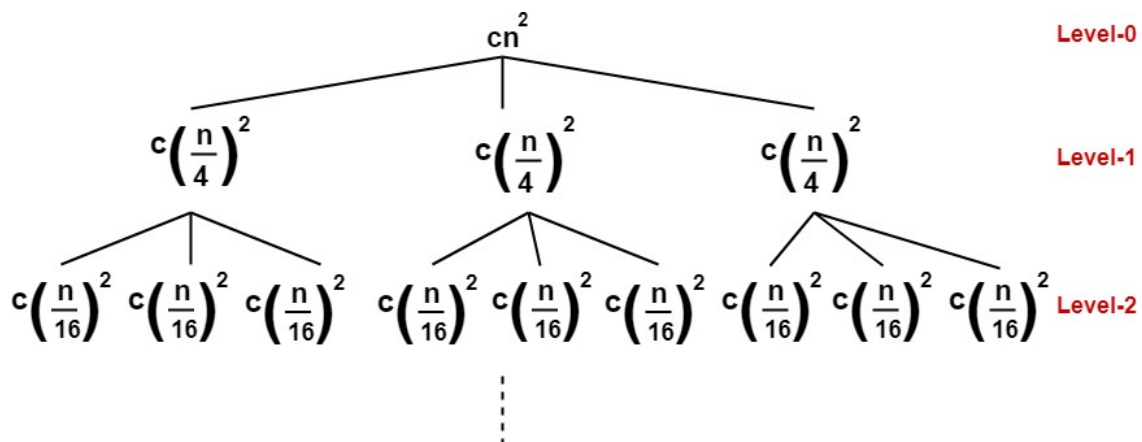
Solve the following recurrence relation using recursion tree method-

$$T(n) = 3T(n/4) + cn^2$$

Solution-

Step-01:

Draw a recursion tree based on the given recurrence relation-





BRAINWARE UNIVERSITY

[PCC-CSM402]

[Design and Analysis of Algorithm]

(Here, we have directly drawn a recursion tree representing the cost of sub problems)

Step-02:

Determine cost of each level-

- Cost of level-0 = cn^2
- Cost of level-1 = $c(n/4)^2 + c(n/4)^2 + c(n/4)^2 = (3/16)cn^2$
- Cost of level-2 = $c(n/16)^2 \times 9 = (9/16^2)cn^2$

Step-03:

Determine total number of levels in the recursion tree-

- Size of sub-problem at level-0 = $n/4^0$
- Size of sub-problem at level-1 = $n/4^1$
- Size of sub-problem at level-2 = $n/4^2$

Continuing in similar manner, we have-

$$\text{Size of sub-problem at level-}i = n/4^i$$

Suppose at level-x (last level), size of sub-problem becomes 1. Then-

$$n/4^x = 1$$

$$4^x = n$$

Taking log on both sides, we get-

$$x \log 4 = \log n$$

$$x = \log_4 n$$

$$\therefore \text{Total number of levels in the recursion tree} = \log_4 n + 1$$



BRAINWARE UNIVERSITY

[PCC-CSM402]

[Design and Analysis of Algorithm]

Step-04:

Determine number of nodes in the last level-

- Level-0 has 3^0 nodes i.e. 1 node
- Level-1 has 3^1 nodes i.e. 3 nodes
- Level-2 has 3^2 nodes i.e. 9 nodes

Continuing in similar manner, we have-

Level- $\log_4 n$ has $3^{\log_4 n}$ nodes i.e. $n^{\log_4 3}$ nodes

Step-05:

Determine cost of last level-

Cost of last level = $n^{\log_4 3} \times T(1) = \theta(n^{\log_4 3})$

Step-06:

Add costs of all the levels of the recursion tree and simplify the expression so obtained in terms of asymptotic notation-

$$T(n) = \underbrace{\left\{ cn^2 + \frac{3}{16} cn^2 + \frac{9}{(16)^2} cn^2 + \dots \right\}}_{\text{For } \log_4 n \text{ levels}} + \theta(n^{\log_4 3})$$

$$= cn^2 \{ 1 + (3/16) + (3/16)^2 + \dots \} + \theta(n^{\log_4 3})$$



BRAINWARE UNIVERSITY

[PCC-CSM402]

[Design and Analysis of Algorithm]

Now, $\{ 1 + (3/16) + (3/16)^2 + \dots \}$ forms an infinite Geometric progression.

On solving, we get-

$$= (16/13)cn^2 \{ 1 - (3/16)^{\log_4 n} \} + \theta(n^{\log_4 3})$$

$$= (16/13)cn^2 - (16/13)cn^2 (3/16)^{\log_4 n} + \theta(n^{\log_4 3})$$

$$= \mathbf{O(n^2)}$$